Načrt za testni projekt

Nikita Jovanov 10/06/2026

1. Tehnologije

Stack:

- React + TypeScript (Vite, PWA): Ena spletna aplikacija, prilagojena telefonu, ki se namesti na Android in deluje tudi na računalniku. (Omogoča kamero in delo brez interneta)
- Node + Express + TypeScript: Isti jezik na strežniku in v aplikaciji, zato lahko tipe podatkov uporabim na obeh straneh.
- PostgreSQL, with Prisma as the ORM): Postgress je tu boljši od MongoDBja ker Podatki so med seboj tesno povezani, zato relacijska baza omogoča preporste preglede in seštevanje ur, prisma pa poskrbi za tipe in migracije.

2. Struktura baze

PostgreSQL, relacijska baza. Vsi ID-ji so UUID, ki jih lahko ustvari že telefon, kar potrebujem za offline sinhronizacijo.

Tabela	Glavna polja
users	id, ime, email, geslo (hash), vloga (DELAVEC / VODJA)
projects	id, ime (npr. Hiša Novak), objekt/lokacija
work_orders	id, številka (2026-00125), project_id, delavec_id, vodja_id, opis, planirana količina in enota, planirane ure, rok, status (NOV / V DELU / ZAKLJUČEN / PROBLEM) Več delavcih pri enem naročilu, bi bilo treba dodati povezovalno tabelo.
work_orders_materials	id, work_order_id, naziv, uporabljena količina
time_entries	id, user_id, work_order_id, začetek, konec (ure se izračunajo iz razlike)
work_logs	id, work_order_id, user_id, opravljeno delo, količina, enota, opomba/problem, čas vnosa
photos	id, work_order_id, user_id, vrsta (PRED / MED / KONČANO / PROBLEM), pot do datoteke, čas posnetka
daily_reports	id, user_id, datum, status (osnutek / oddano), čas oddaje

3. Izgled aplikacije

Aplikacija je narejena najprej za telefon: velike tipke, ena glavna akcija na zaslon, spodnja navigacija. Odpre se prek povezave, brez nameščanja iz trgovine. Zasloni so na skicah spodaj.

Delavec: prijava, seznam nalogov po objektih s statusom, delovni nalog (začni nalog, začni in končaj delo, vnos dela, fotografija, status Problem, zaključi nalog) in dnevno poročilo.

Vodja: pregled s filtrom (na računalniku tabela, na telefonu seznam kartic), podrobnosti naloga in seznam projektov.

4. Delitev 40 ur

- Dan 1: priprava projekta, baza, prijava, projekti in delovni nalogi
- Dan 2: začetek in konec dela z izračunom ur, vnos opravljenega dela, količine in problemov
- Dan 3: fotografije in dnevno poročilo
- Dan 4: pregled za vodjo, podrobnosti naloga, testni podatki in prilagoditev telefonu
- Dan 5: popravki, PDF poročilo, glasovni vnos in priprava predstavitve

Vsak dan ima 8 ur. Če katera točka vzame več časa, se premakne bonus, osnovni potek pa ostane.

5. Katere funkcionalnosti najprej

Najprej naredim glavni potek, ki ga želite videti na predstavitvi, od prijave do pregleda vodje. Vsako stvar naredim do konca, od zaslona do baze, da imam ves čas nekaj, kar dejansko dela. Potem naredim PDF, ker so podatki zanj že v dnevnem poročilu, in glasovni vnos, ki zapiše govor v polje za opombo. Offline ne bom izdelal, bom pa aplikacijo že od začetka naredil tako, da ga je mogoče dodati: ID-je in čas vnosov ustvari telefon, strežnik pa že poslane vnose prezre.

6. Offline v končni aplikaciji

Aplikacijo bi naredil kot PWA. Service worker z Workboxom shrani aplikacijo na telefon, zato se odpre tudi brez interneta. Ob prijavi se delavčevi odprti nalogi shranijo v IndexedDB prek knjižnice Dexie. Če ni signala, delavec dela naprej, vsi vnosi pa gredo v lokalno vrsto na telefonu. Vsak vnos dobi UUID in čas že na telefonu, fotografije pa se shranijo kot Blob in naložijo pozneje. Ko se povezava vrne, aplikacija vrsto pošlje strežniku, ki že prejete zapise prezre, da se ure ne podvojijo. Konfliktov skoraj ni, ker vsak ureja samo svoje vnose. V aplikaciji se vidi, kaj še čaka na pošiljanje.

Hiter prototip

